

MODULE :- Unit Test Frameworks

EXPERIMENT TITLE :- pytest Integration with HTML Reporting (Assignment 9)

1.Problem Statement :- To automate web application test cases using **Selenium with pytest**, implement **fixtures for setup and teardown**, use the **Page Object Model**, and generate an **HTML test report**.

2. Tools and Software Required :-

S. No.	Tool / Software	Purpose
1	Python	Programming language used to write the automation scripts
2	PyCharm / VS Code	IDE used to write and execute Python code
3	Google Chrome	Web browser used for automation
4	Selenium WebDriver	Automates the browser and interacts with web elements
5	pytest	Testing framework used to create and execute test cases
6	pytest-html	Generates HTML test execution reports
7	Command Prompt / Terminal	Used to install packages and run pytest commands
8	Test Automation Practice Website	Website used as the application under test

3. Concepts Used :-

Selenium WebDriver is used to automate and interact with the web browser.

pytest is used to create and execute automated test cases.

Fixtures are used for browser setup and teardown.

Assertions are used to verify whether the actual result matches the expected result.

Page Object Model (POM) is used to organize web-element locators and page actions.

HTML Reporting is used to generate a report showing the test execution results.

4. Source Code :-

File 1: conf test .py

```
import pytest
from selenium import webdriver

# Fixture for browser setup and teardown
@pytest.fixture
def driver():

    # -----
    # SETUP
    # -----
    driver = webdriver.Chrome()

    driver.maximize_window()

    driver.get(
        "https://testautomationpractice.blogspot.com/2018/09/automation-form.html"
    )

    # Give driver to the test case
    yield driver

    # -----
    # TEARDOWN
    # -----
    driver.quit()
```

File 2: test_automation.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

# =====
# PAGE OBJECT CLASS
# =====

class AutomationPage:
```

```

# -----
# Locators
# -----

name = (
    By.CSS_SELECTOR,
    "input[placeholder='Enter Name']"
)

email = (
    By.CSS_SELECTOR,
    "input[placeholder='Enter EMail']"
)

phone = (
    By.CSS_SELECTOR,
    "input[placeholder='Enter Phone']"
)

address = (
    By.TAG_NAME,
    "textarea"
)

# -----
# Constructor
# -----

def __init__(self, driver):
    self.driver = driver

# -----
# Page Actions
# -----

def enter_name(self, value):
    self.driver.find_element(*self.name).send_keys(value)

```

```

def enter_email(self, value):
    self.driver.find_element(*self.email).send_keys(value)

def enter_phone(self, value):
    self.driver.find_element(*self.phone).send_keys(value)

def enter_address(self, value):
    self.driver.find_element(*self.address).send_keys(value)

# =====
# TEST CASE 1
# Check Page Title
# =====

def test_page_title(driver):

    assert "Automation Testing Practice" in driver.title

# =====
# TEST CASE 2
# Enter Data in Form
# =====

def test_enter_form_data(driver):

    # Create Page Object
    page = AutomationPage(driver)

    # Enter details
    page.enter_name("Prateek Sinha")

    page.enter_email("prateek@gmail.com")

    page.enter_phone("9876543210")

```

```

page.enter_address("Kolkata, India")

# Assertions
assert driver.find_element(
    *page.name
).get_attribute("value") == "Prateek Sinha"

assert driver.find_element(
    *page.email
).get_attribute("value") == "prateek@gmail.com"

# =====
# TEST CASE 3
# Radio Button and Checkbox
# =====

def test_radio_and_checkbox(driver):

    # Find radio buttons
    radio_buttons = driver.find_elements(
        By.CSS_SELECTOR,
        "input[type='radio']"
    )

    # Select first radio button
    radio_buttons[0].click()

    # Find checkboxes
    checkboxes = driver.find_elements(
        By.CSS_SELECTOR,
        "input[type='checkbox']"
    )

    # Select first checkbox
    checkboxes[0].click()

    # Assertions

```

```

    assert radio_buttons[0].is_selected()

    assert checkboxes[0].is_selected()

# =====
# TEST CASE 4
# Country Dropdown
# =====

def test_country_dropdown(driver):

    # Find all dropdowns
    dropdowns = driver.find_elements(
        By.TAG_NAME,
        "select"
    )

    # Select first dropdown
    country = Select(dropdowns[0])

    country.select_by_visible_text("India")

    # Assertion
    assert country.first_selected_option.text == "India"

```

5. Implementation Steps :-

1. Install **Python, Selenium, PyTest, and pytest-html**.
2. Create `conf test.py` for **browser setup and teardown**.
3. Create `test_automation.py` for the **Page Object and test cases**.
4. Open the **Test Automation Practice** website using Selenium.
5. Perform form actions such as entering data, selecting radio buttons, checkboxes, and dropdowns.
6. Use **assertions** to verify the test results.

- 7.Run all test cases using PyTest.
- 8.Generate an **HTML test report** using `pytest-html`.
- 9.Verify the test results in the generated report.

6.Output :-

```
PS C:\Users\catch\OneDrive\Desktop\wipro_assignments\assignment_9> pytest -v
collected 4 items

test_automation.py::test_page_title PASSED [ 25%]
test_automation.py::test_enter_form_data PASSED [ 50%]
test_automation.py::test_radio_and_checkbox PASSED [ 75%]
test_automation.py::test_country_dropdown PASSED [100%]

===== 4 passed in 28.48s =====
PS C:\Users\catch\OneDrive\Desktop\wipro_assignments\assignment_9> pytest -v --html=report.html --self-contained-html
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\catch\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.13.5', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'anyio': '4.12.1', 'langsmith': '0.7.9', 'html': '4.2.0', 'metadata': '3.1.1'}, 'JAVA_HOME': 'C:\\Program Files\\Eclipse Adoptium\\jdk-17.0.13.11-hotspot\\'}
rootdir: C:\Users\catch\OneDrive\Desktop\wipro_assignments\assignment_9
plugins: anyio-4.12.1, langsmith-0.7.9, html-4.2.0, metadata-3.1.1
collected 4 items

test_automation.py::test_page_title PASSED [ 25%]
test_automation.py::test_enter_form_data PASSED [ 50%]
test_automation.py::test_radio_and_checkbox PASSED [ 75%]
test_automation.py::test_country_dropdown PASSED [100%]

----- Generated html report: file:///C:/Users/catch/OneDrive/Desktop/wipro_assignments/assignment_9/report.html -----
===== 4 passed in 26.63s =====
PS C:\Users\catch\OneDrive\Desktop\wipro_assignments\assignment_9>
```

report.html

Report generated on 25-Sep-2026 at 19:51:08 by `pytest-html` v4.2.0

Environment

Python	3.13.5
Platform	Windows-11-10.0.26200-SP0
Packages	- pytest: 9.1.1 - pluggy: 1.6.0
Plugins	- anyio: 4.12.1 - langsmith: 0.7.9 - html: 4.2.0 - metadata: 3.1.1
JAVA_HOME	C:\Program Files\Eclipse Adoptium\jdk-17.0.13.11-hotspot\

Summary

4 tests took 00:00:27.

(Un)check the boxes to filter the results.

☒ 0 Failed,	☒ 4 Passed,	☒ 0 Skipped,	☒ 0 Expected failures,	☒ 0 Unexpected passes,	☒ 0 Errors,	☒ 0 Reruns
Result 	Test					
Passed	test_automation.py::test_page_title					
Passed	test_automation.py::test_enter_form_data					
Passed	test_automation.py::test_radio_and_checkbox					
Passed	test_automation.py::test_country_dropdown					

7.Result :-

The Selenium test cases were successfully executed using PyTest, and the HTML report was generated successfully.

8.Observation:-

The web form elements such as text fields, radio buttons, checkboxes, and dropdowns were identified and tested correctly.

9.Conclusion:-

Thus, Selenium with PyTest was successfully used for basic web automation, test execution, assertions, and HTML reporting.